

Reviewer Pack — Minimal Local Defect Complexity of Affine Quotients vs Boole...

Entry 71df4ad88e5f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 06:11:59 UTC

Minimal Local Defect Complexity of Affine Quotients vs Boolean Function Monomial Symmetry

Entry ID: 71df4ad88e5f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 06:11:59 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Affine Quotients) **Field B** (complexity object): Complexity Theory (Boolean Functions)

Statement:

[‘For any n -ary boolean function f , the minimal local defect complexity of its affine quotient is upper bounded by a constant times the number of monomial symmetries in f .’, ‘Formally, for all n -ary boolean functions f with minimal local defect complexity D_f and number of monomial symmetries $S(f)$, we have $D_f \leq c * S(f)$ where c is a constant.’, ‘The conjecture falsifies by providing a boolean function with a high number of monomial symmetries but low minimal local defect complexity.’]

Rationale (proposer’s reasoning):

[‘Affine quotient geometry can be used to study algebraic properties of functions, while the study of monomial symmetry in boolean functions has connections to their computational complexity.’, ‘This conjecture could expose a new connection between algebraic geometric properties and the symmetries of boolean functions, potentially leading to new insights into function complexity.’]

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dd8e7d3c3b337a1e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 90% of the n -ary boolean functions generated, the ratio of the minimal local defect complexity to the number of monomial symmetries does not exceed a constant c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def count_monomial_symmetries(f):
        n = int(math.log2(len(f)))
        symmetries = set()
        for i in range(2**n):
            permuted_f = [f[(i >> j) & 1] for j in range(n)]
            if f == permuted_f:
                symmetries.add(tuple(permuted_f))
        return len(symmetries)

    def compute_minimal_local_defect_complexity(f):
        n = int(math.log2(len(f)))
        max_defect = 0
        for i in range(2**n):
            defect = sum(abs(f[i] - f[j]) for j in range(i, 2**n) if (i ^ j).bit_count() == 1)
            if defect > max_defect:
                max_defect = defect
        return max_defect

    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    S_f = count_monomial_symmetries(f)
    D_f = compute_minimal_local_defect_complexity(f)

    if S_f == 0:
```

```

    return {
        "metric_name": "ratio",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "No monomial symmetries"
    }

ratio = D_f / S_f
return {
    "metric_name": "ratio",
    "metric_value": ratio,
    "instances_tested": 1,
    "conjecture_holds": True if ratio <= 10 else False, # Placeholder constant for demonstration
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='High ratio' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture with the provided evidence. | next: Run the test again to ensure it completes successfully and provides results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11910
2	propose	ollama_remote	glm4:latest	0	0	10364
3	preregistration	ollama_remote	glm4:latest	0	0	5655
4	novelty	ollama_remote	glm4:latest	0	0	4925
5	novelty	ollama_remote	glm4:latest	0	0	5214
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14898
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10314
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6672
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9065
10	judge	ollama_remote	glm4:latest	0	0	13607

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 92625 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/71df4ad88e5f.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/71df4ad88e5f.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/71df4ad88e5f.tar.gz (if generated)